01. System Integration & Architecture Guide

1. Executive Summary

The **Network Sensing Layer** is Station 1 of the Modular SIEM (Security Information and Event Management) platform. Its role is **passive, non-intrusive traffic inspection and telemetry shipping**.

It intercepts network traffic in real time, extracts TCP/IP metadata, reassembles fragmented TLS ClientHellos, computes RFC 8701 compliant **JA4 client fingerprints**, identifies target domain names via Server Name Indication (SNI), caches ongoing flows in memory, and asynchronously ships structured JSON events to the SIEM Ingestion API.

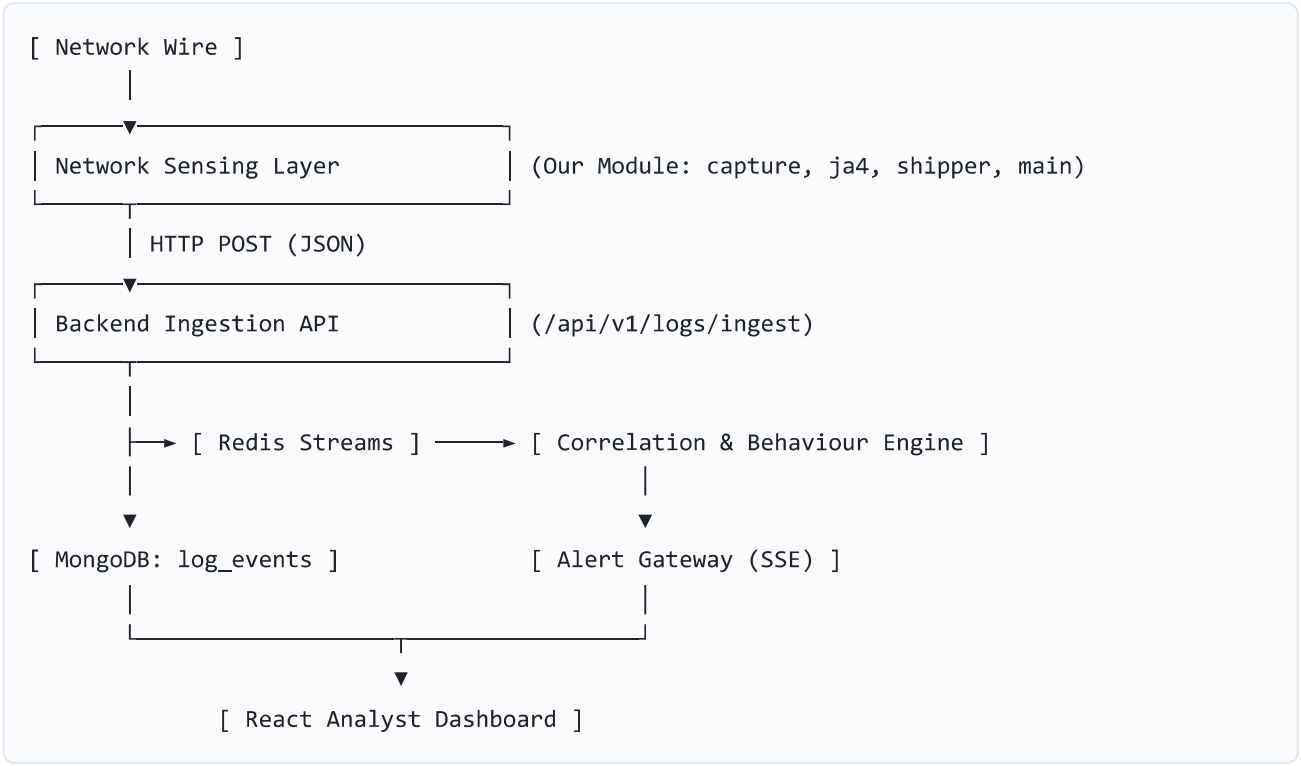
2. End-to-End Sensing Layer Pipeline

The sensing layer is built as a 4-station decoupled pipeline:



3. SIEM Ecosystem Integration Map

To understand where your code fits, here is the full dataflow from the network wire to the analyst's screen:



4. Layer-by-Layer Interaction Analysis

Layer A: Backend Ingestion API (`POST /api/v1/logs/ingest`)

- **Direct Interaction: YES**.
- **How it interacts:** `shipper.py` sends HTTP POST requests containing JSON-serialized metadata.
- **Contract:** The backend receives payloads with `source_type="network"`, validates fields, and pushes them to Redis Streams and MongoDB.

Layer B: Redis Streams (Message Broker)

- **Direct Interaction: NO**.
- **Indirect Role:** The backend ingestion controller receives our HTTP POST and writes the log into a Redis Stream.
- **Purpose:** Redis decouples ingestion from heavy threat correlation engines so network spikes do not choke database writes.

Layer C: Database (MongoDB via MongoEngine `LogEvent`)

- **Direct Interaction: NO**.

- **Indirect Role:** The network sensing layer never talks directly to MongoDB (direct database writes from network probes create database connection exhaustion and bypass security access controls).
- **Field Harmonization:** Every key in our `master_directory` emitted by `ja4.py` maps 1:1 to fields in the backend `LogEvent` document model:
    - `source_type` ➔ `LogEvent.source_type` ("network")
    - `source_ip` ➔ `LogEvent.source_ip`
    - `dest_ip` ➔ `LogEvent.dest_ip`
    - `source_port` ➔ `LogEvent.source_port`
    - `dest_port` ➔ `LogEvent.dest_port`
    - `protocol` ➔ `LogEvent.protocol` ("tcp")
    - `tcp_flags` ➔ `LogEvent.tcp_flags`
    - `packet_size` ➔ `LogEvent.packet_size`
    - `ja4` ➔ `LogEvent.ja4`
    - `metadata.sni` ➔ `LogEvent.metadata["sni"]`
    - `metadata.time_to_live` ➔ `LogEvent.metadata["time_to_live"]`

Layer D: Correlation & Behavioural Engine

- **Direct Interaction: NO**.
- **Indirect Role:** The correlation engine consumes the network logs from Redis. It evaluates:
    1. **JA4 Threat Intel Feeds:** Compares `ja4` against known malware fingerprints (e.g., Cobalt Strike, Metasploit, PoshC2, TrickBot).
    2. **DGA / DNS Anomaly:** Compares `dest_ip` and `metadata.sni` against domain reputation databases.
    3. **Port Scanning / Flooding:** Counts distinct `dest_port` occurrences per `source_ip` using sliding time windows.

Layer E: Frontend (React Analyst Dashboard)

- **Direct Interaction: NO**.
- **Indirect Role:** Analysts query logs and alerts through the React UI.
    - The UI displays live network telemetry tables showing `Source IP`, `Destination IP`, `SNI`, and `JA4 Fingerprint`.
    - Analysts can click a JA4 hash to view all sessions across the enterprise that share that cryptographic client profile.

5. What This Layer Intentionally Does NOT Do

1. **No Payload Decryption:** We inspect metadata only (IP headers, TCP headers, and unencrypted TLS ClientHello bytes). We do not decrypt HTTPS traffic, ensuring user privacy and regulatory compliance.
2. **No Direct Database Access:** We do not import MongoDB/MongoEngine or store database credentials on probe machines. All logs route through authenticated REST endpoints.
3. **No Alert Generation:** Alerts are the job of the Correlation Engine. The sensor's sole responsibility is high-speed, faithful telemetry capture.